



University
of Glasgow

A WORLD
TOP 100
UNIVERSITY

Introduction to Data Science and Systems

Lecture 5: **Optimisation II**

Dr Nicolas Pugeault

WORLD
CHANGING
GLASGOW

THE SUNDAY TIMES
GOOD
UNIVERSITY
GUIDE
2024
SCOTTISH
UNIVERSITY
OF THE YEAR



University
of Glasgow

Deep neural networks





Deep neural networks

- **Deep learning** or **deep neural networks** have become a major part of modern machine learning research.
- They have had astonishing success in fields like speech recognition, machine translation, image classification and image synthesis.
- The basic problem of deep learning is one of finding an approximating function.

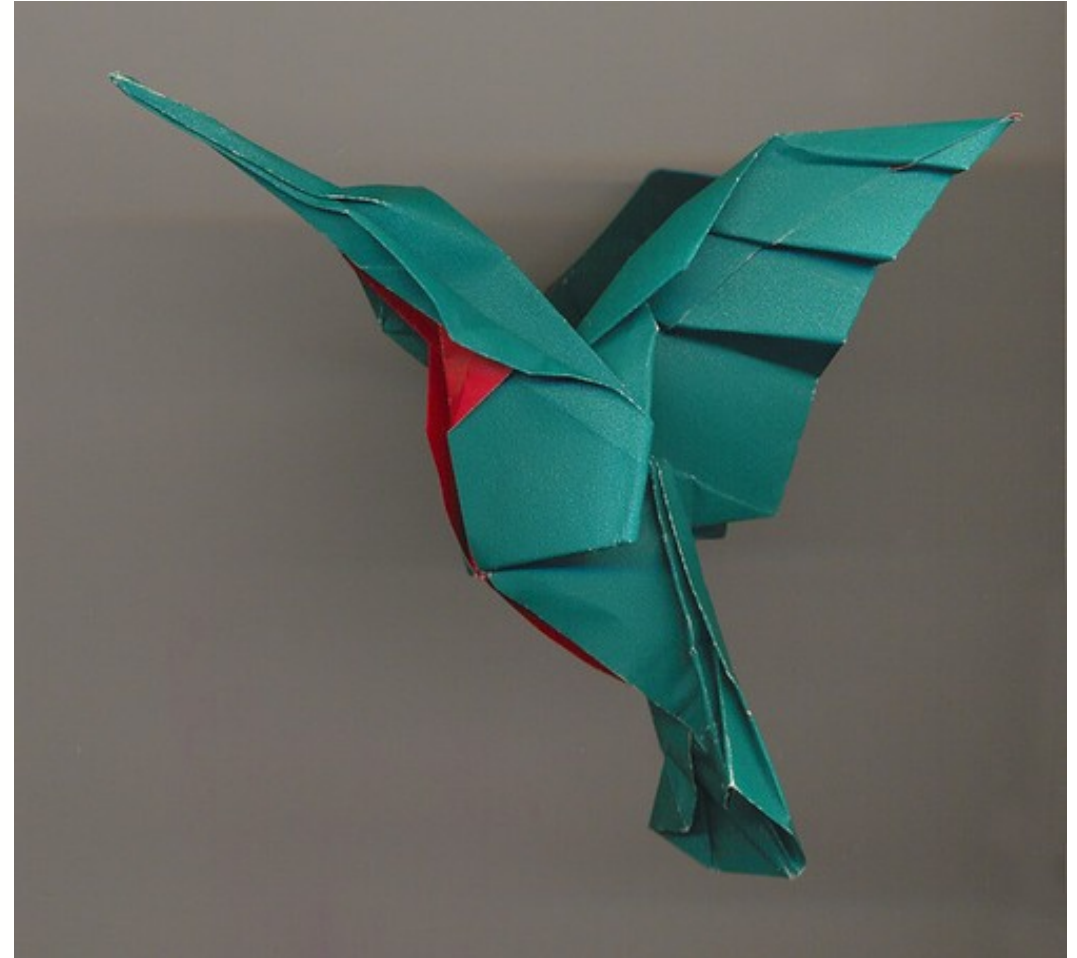
$$\theta^* = \arg \min_{\theta} \sum_i ||f(\mathbf{x}_i; \theta) - y_i||$$



Deep Neural Networks as Origami

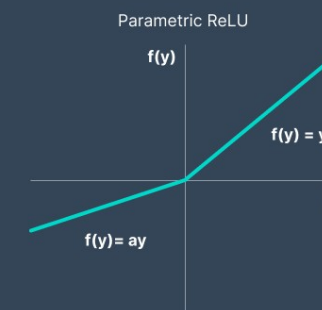
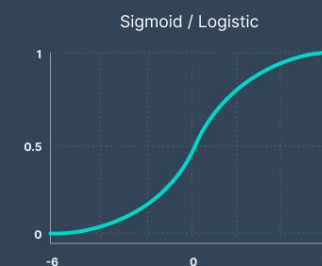
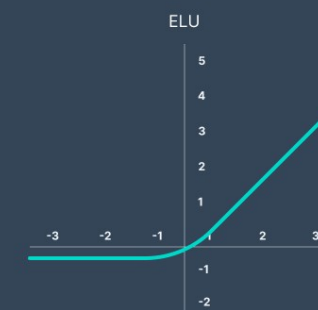
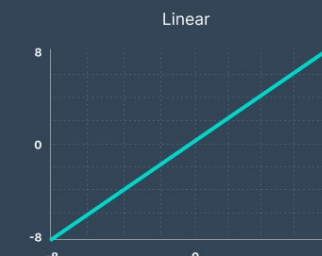
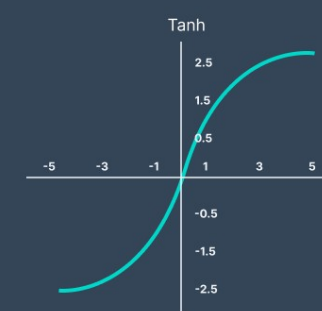
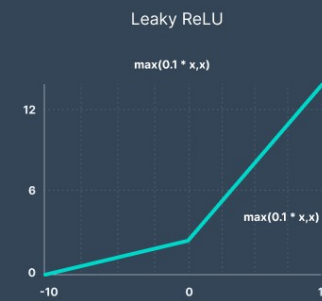
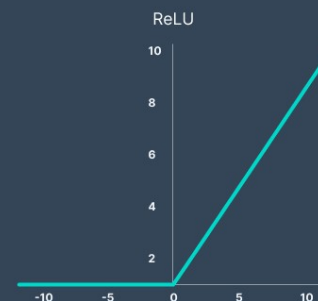
- A linear map (matrix multiplication), that rotates, translates the input (“paper”)
- A nonlinear function that squashes the input (“folds”)
- Formally:

$$y_i = G(W_i x_i + b_i)$$



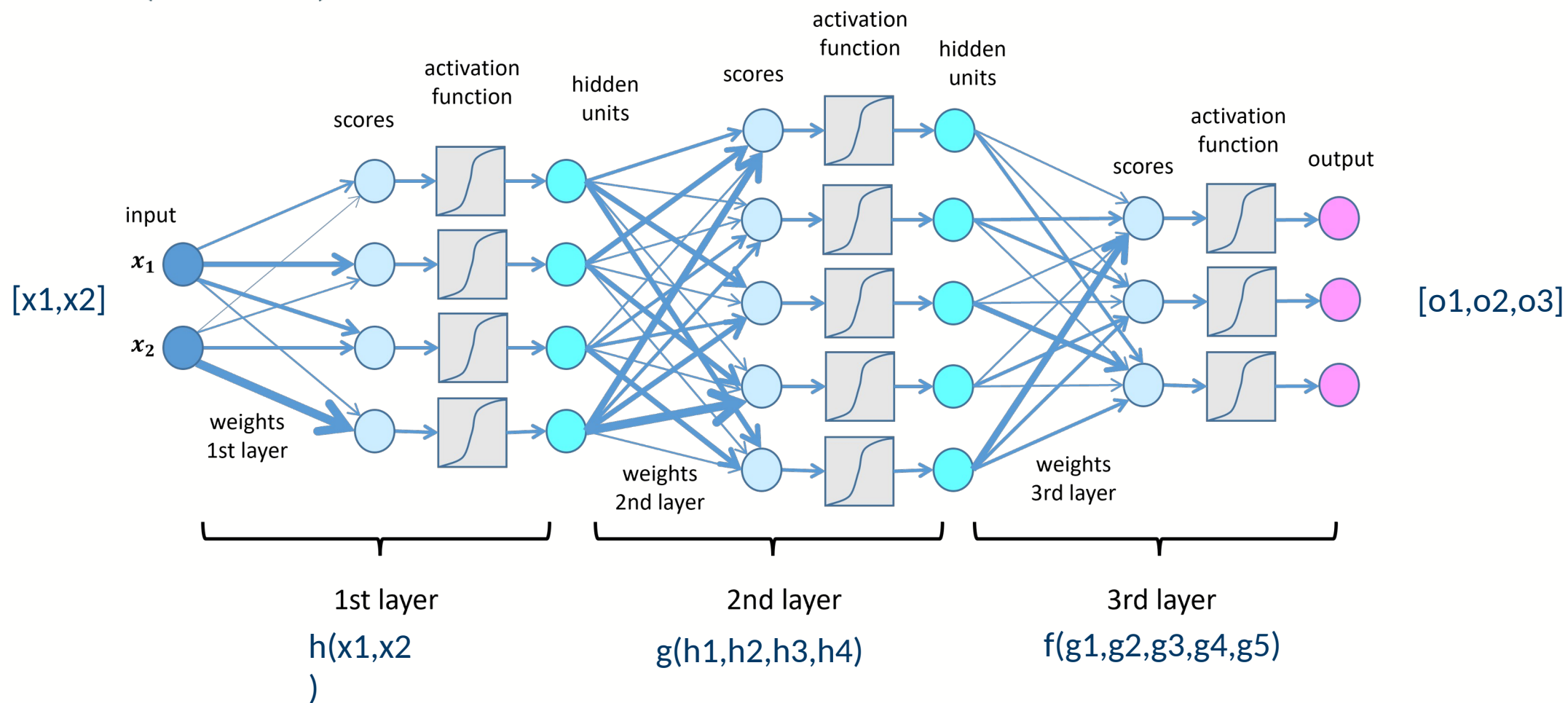


- **nonlinear (activation) function** $G(x)$
- **nonlinear functions** are essential in neural networks because they allow the network to model complex, real-world data by introducing **nonlinearity**.
- Without them, a neural network would be limited to solving simple **linear problems**, no matter how deep or wide the network is.



A 3-layer deep neural network (DNN)

- DNN: $f(g(h(x)))$





Why not use heuristic search?

- Heuristic search methods like random search, simulated annealing and genetic algorithms are easy to understand, easy to implement and have few restrictions on the problems they can be applied to. **However** ...
 - They can be very slow; it may take many iterations to approach a minimum and require significant computation to compute each iteration.
 - There is no guarantee of convergence, or even of progress. The search can get stuck, or drift slowly over plateaus.
 - There are a huge number of **hyperparameters** that can be tweaked (temperature schedules, size of population, memory structure, etc.). How should these be chosen? Optimal choice of these parameters becomes an optimisation problem in itself.



Why not use heuristic search?

- For optimisation problems like **deep neural networks**, heuristic search is hopelessly inadequate. -- 1000b parameters
- Instead, **first-order** optimisation is applied.
- **First-order algorithms**, that we will discuss today, can be *orders of magnitude* faster than heuristic search.

Gravity applies a force perpendicular to the plane of the surface. The surface applies a force to the ball along the direction of the *surface normal*, the vector that points "straight out" from the surface. This results in a force component in *the direction of the steepest slope* of the surface, accelerating the ball in that direction



The derivative of a scalar function

- How do we find minimum of a scalar function?

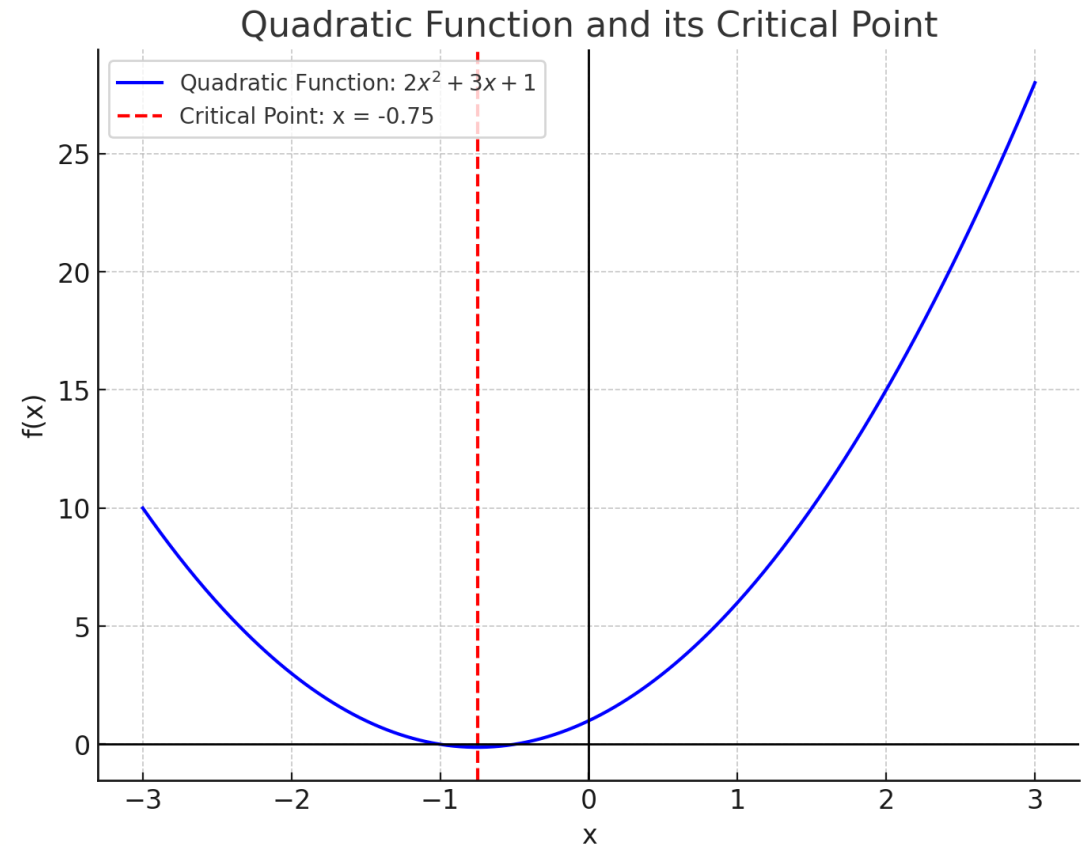
$$f(x) = 2x^2 + 3x + 1$$

- We find where the derivative is zero:

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \mathbf{0}$$

- Derivative:

$$f'(x) = 4x + 3 = 0$$
$$x = -\frac{3}{4}$$





The derivative of a scalar function

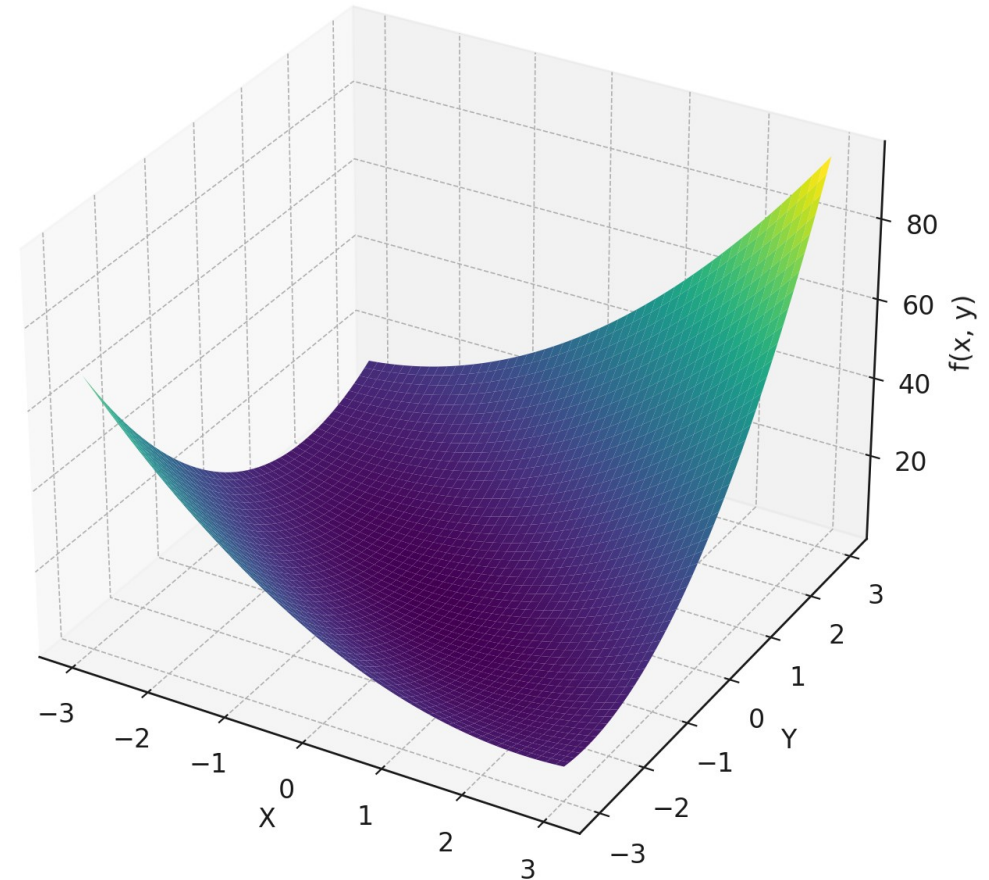
And for a function of two variables?

$$f(x, y) = 2x^2 + 3y^2 + 4xy + 2x + 3y + 1$$

We need to set the derivative of the function to zero *with respect to each variable*:

$$\frac{\partial f(x)}{\partial x} = 4x + 4y + 2 = 0$$

$$\frac{\partial f(y)}{\partial y} = 6y + 4x + 3 = 0$$



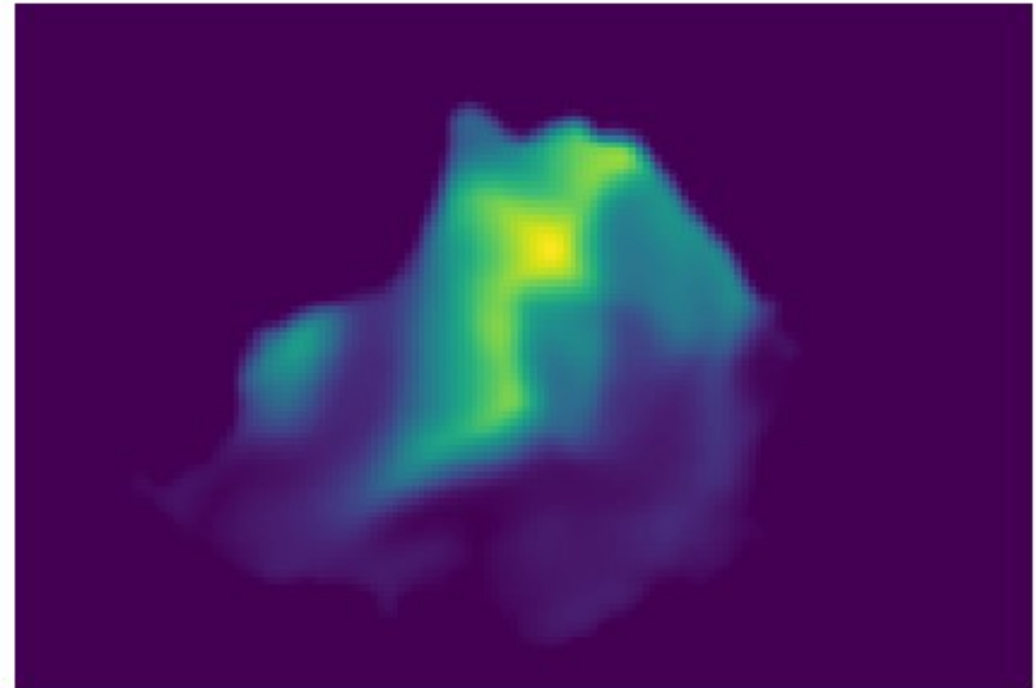


The derivative of a scalar function

The derivative of a **scalar** function
 $f(\mathbf{x}): \mathbb{R}^N \rightarrow \mathbb{R}$ with respect to vector
 $\mathbf{x} \in \mathbb{R}^N$

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \triangleq \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_N} \end{bmatrix}$$

Function

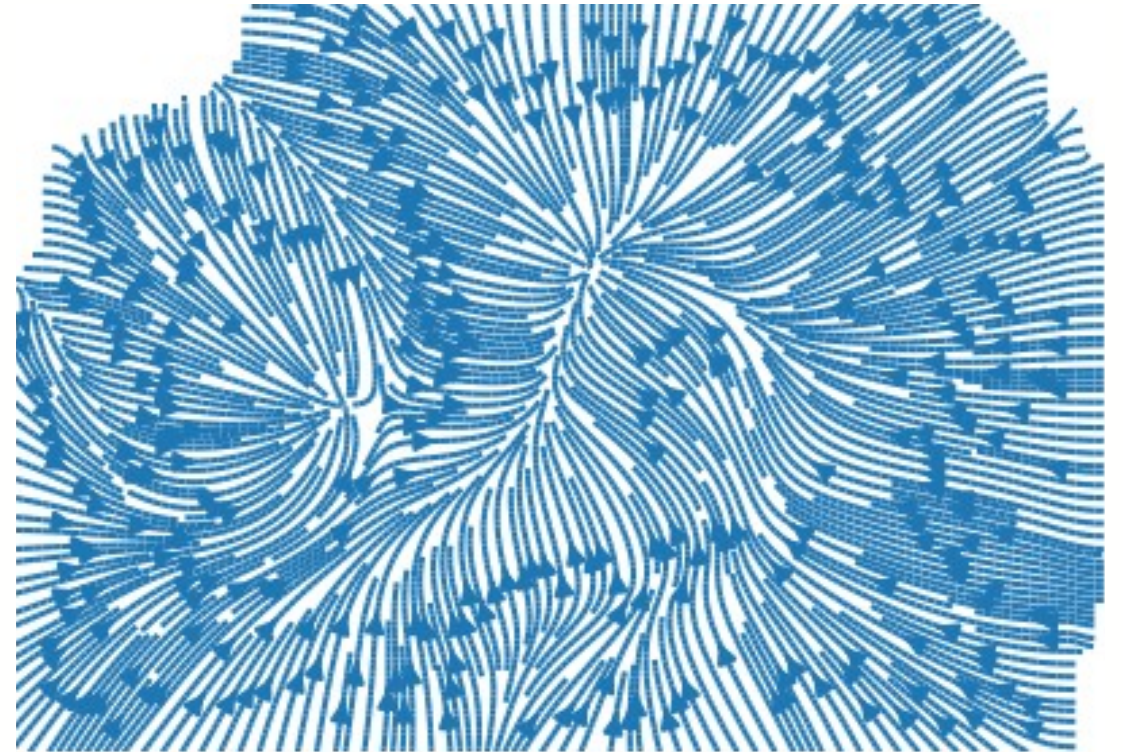




The derivative of a scalar function

- Derivative of a **vector** function $\mathbf{f}(\mathbf{x}): \mathbb{R}^N \rightarrow \mathbb{R}^M$ with respect to vector $\mathbf{x} \in \mathbb{R}^N$.

$$\frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} \triangleq \begin{bmatrix} \frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_1} \\ \frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_N} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \frac{\partial f_2(\mathbf{x})}{\partial x_1} & \cdots & \frac{\partial f_M(\mathbf{x})}{\partial x_1} \\ \frac{\partial f_1(\mathbf{x})}{\partial x_2} & \frac{\partial f_2(\mathbf{x})}{\partial x_2} & \cdots & \frac{\partial f_M(\mathbf{x})}{\partial x_2} \\ \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_1(\mathbf{x})}{\partial x_N} & \frac{\partial f_2(\mathbf{x})}{\partial x_N} & \cdots & \frac{\partial f_M(\mathbf{x})}{\partial x_N} \end{bmatrix}$$





Jacobian: matrix of derivatives

- We can collect this derivative information into a matrix called the **Jacobian matrix**, which characterises the slope *at a specific point* $\mathbf{x}$.

$$f'(\mathbf{x}) = \mathbf{J} = \begin{bmatrix} \frac{\partial y_0}{\partial x_0} & \frac{\partial y_0}{\partial x_1} & \cdots & \frac{\partial y_0}{\partial x_n} \\ \frac{\partial y_1}{\partial x_0} & \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_0} & \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

- This simply tells us how much each **component** of the output changes as we change any component of the input -- the generalised "slope" of a vector-valued function.



- $\nabla f(x)$ is the *gradient vector* of **a (scalar) function** of a vector, the equivalent of the first derivative for vector functions.
- This tells us how much $f(x)$ would vary if we made tiny changes to each dimension *independently*.
- We will work with **scalar** objective functions $L(\theta)$ of parameter vectors θ .

$$\nabla L(\theta) = \left[\frac{\partial L(\theta)}{\partial \theta_1}, \frac{\partial L(\theta)}{\partial \theta_2}, \dots, \frac{\partial L(\theta)}{\partial \theta_n} \right]$$

- $\nabla L(\theta)$ is a vector *which points in the direction of the steepest change in $L(\theta)$* .



- $\nabla^2 L(\theta)$ is the **Hessian matrix** of a (scalar) function of a vector, the equivalent of the **second** derivative for vector functions.
- Following our rule above, it's just the Jacobian of a **vector** valued function, so we know:

$\nabla^2 L(\theta)$ is matrix valued map $\mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}$

$$H(L) = \nabla \nabla L(\theta) = \nabla^2 L(\theta) = \begin{bmatrix} \frac{\partial^2 L(\theta)}{\partial \theta_1^2} & \frac{\partial^2 L(\theta)}{\partial \theta_1 \partial \theta_2} & \frac{\partial^2 L(\theta)}{\partial \theta_1 \partial \theta_3} & \cdots & \frac{\partial^2 L(\theta)}{\partial \theta_1 \partial \theta_n} \\ \frac{\partial^2 L(\theta)}{\partial \theta_2 \partial \theta_1} & \frac{\partial^2 L(\theta)}{\partial \theta_2^2} & \frac{\partial^2 L(\theta)}{\partial \theta_2 \partial \theta_3} & \cdots & \frac{\partial^2 L(\theta)}{\partial \theta_2 \partial \theta_n} \\ \cdots & & & & \\ \frac{\partial^2 L(\theta)}{\partial \theta_n \partial \theta_1} & \frac{\partial^2 L(\theta)}{\partial \theta_n \partial \theta_2} & \frac{\partial^2 L(\theta)}{\partial \theta_n \partial \theta_3} & \cdots & \frac{\partial^2 L(\theta)}{\partial \theta_n^2} \end{bmatrix},$$

Differentiable objective functions

- For some objective functions, we can compute the (**exact**) derivatives of the objective function with respect to the parameters θ .
 - For example, if the objective function has a single scalar parameter $\theta \in \mathbb{R}$ and the function is:

$$L(\theta) = \theta^2$$

then, from basic calculus, the derivative with respect to θ is just:

$$L'(\theta) = 2\theta.$$

- If we know the derivative, we can use this to move in "good directions" -- down the slope of the objective function towards a minimum.
- This becomes slightly more involved for multidimensional **objective functions** (where θ has more than one component) where we have a gradient vector instead of a simple scalar derivative (written $\nabla L(\theta)$).



Orders: zeroth, first, second

Iterative algorithms can be classified according to the order of derivative they require:

- a **zeroth order optimisation** algorithm only requires evaluation of the objective function $L(\theta)$. Examples include random search and simulated annealing.
- a **first order optimisation** algorithm requires evaluation of $L(\theta)$ and its derivative $\nabla L(\theta)$. This class includes the family of gradient descent methods.
- a **second order optimisation** algorithm requires evaluation $L(\theta)$, $\nabla L(\theta)$ and $\nabla \nabla L(\theta)$. These methods include quasi-Newtonian optimisation.



Optimisation with derivatives

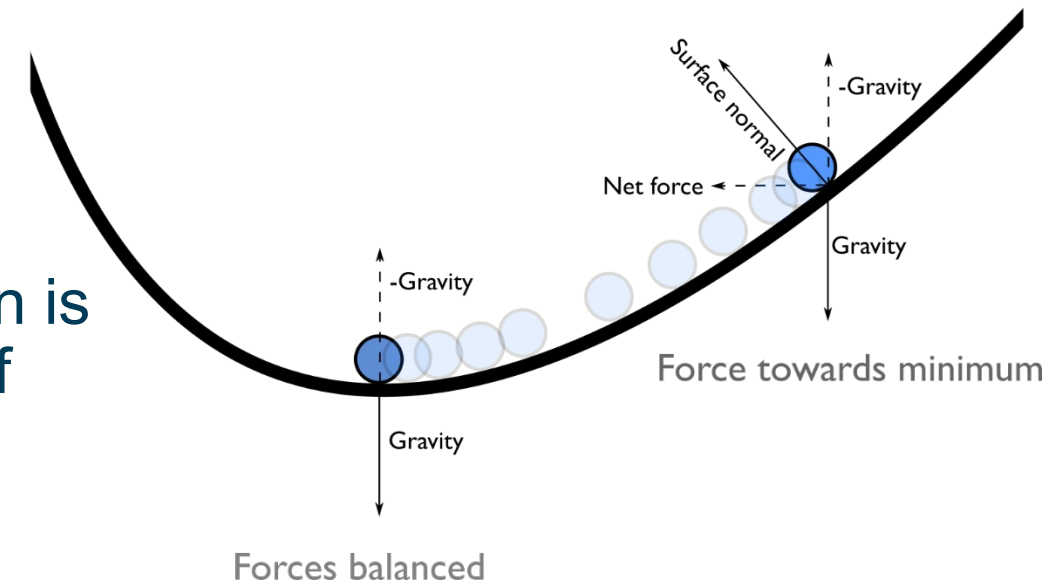
If we know (or can compute) the **gradient** an objective function, we know the **slope** of the function at any given point. This gives us both:

- the direction of fastest increase and
- the steepness of that slope.

This is *the* major application of calculus.

Knowing the **derivative** of the objective function is sufficient to dramatically improve the efficiency of optimisation.

Physical optimisation
(ignoring momentum)

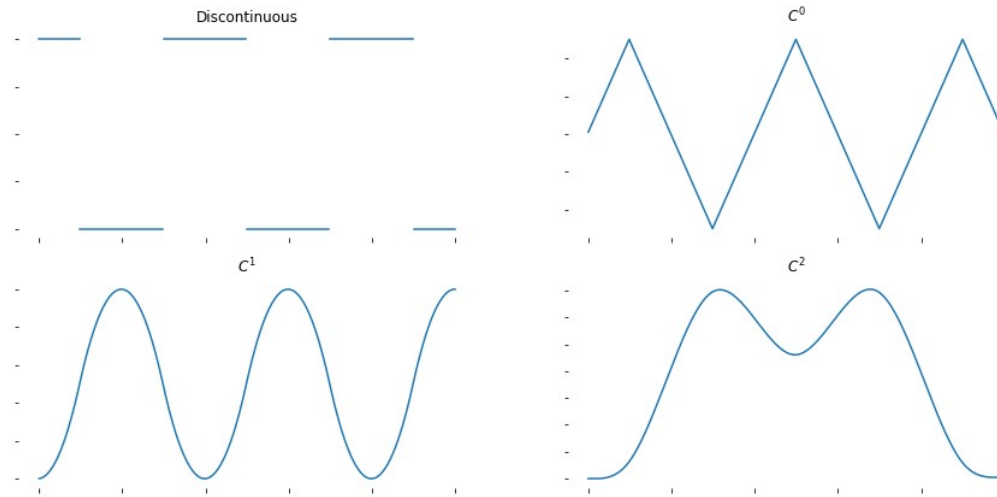


This gradient vector always points in the direction of the steepest slope.



Differentiability

- A **smooth function** has continuous derivatives up to some order. **Smoother** functions are typically **easier** to do iterative optimisation on, because **small** changes in the current approximation are likely to lead to **small** changes in the objective function.
- We say a function is C^n **continuous** if the n th derivative is continuous (**smoothness class**).



*Image: left-to-right, top-to-bottom
discontinuous, C^0 , C^1 , C^2 continuous functions*



continuous derivatives vs differentiable

- There is a difference between *having continuous derivatives* and *knowing what those derivatives are*.
- First order optimisation uses the (first) derivatives of the objective function with respect to the parameters. These techniques can only be applied if the objective function is:
 - At least **continuous** i.e. no step changes anywhere in the function or its derivative
 - **differentiable (Analytically we can derive the derivative)** i.e. gradient is defined everywhere

(though we will see that these constraints can be relaxed a bit in practice).

- Many objective functions satisfy these conditions, and first-order methods can be vastly more efficient than zeroth-order methods. For particular classes of functions (e.g. convex) there are known bounds on number of steps required to converge for specific first-order optimizers.

Lipschitz Continuity

First (and higher) order optimization require a loss function $L(\theta): \mathbb{R}^n \rightarrow \mathbb{R}$ to be **Lipschitz continuous**:

- its **gradient is bounded** and the function cannot change more quickly than some constant.

In other words, there is a maximum steepness:

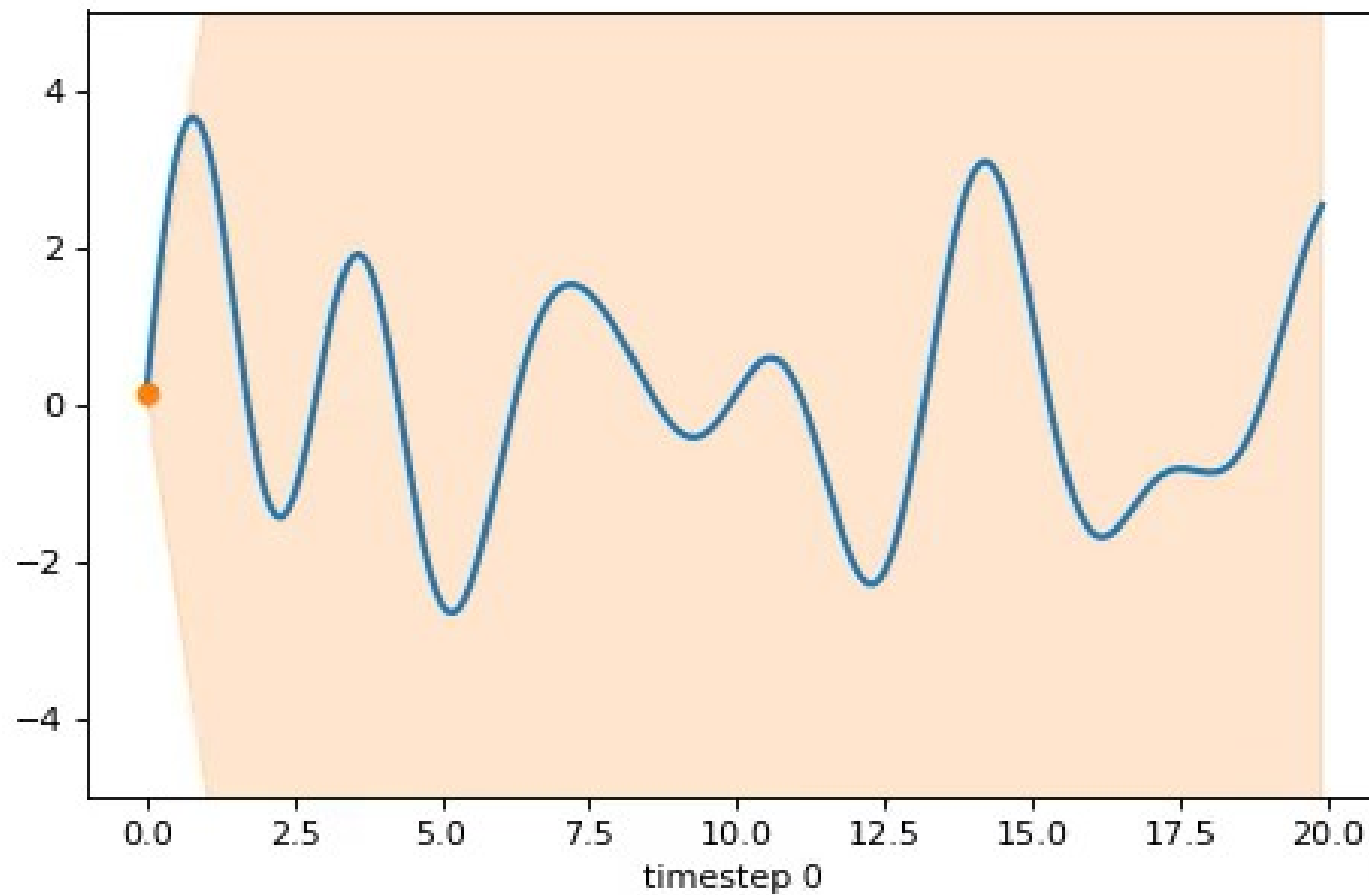
$$\frac{\partial L(\theta)}{\partial \theta_i} < K \text{ for all } i \text{ and some } K$$

- K is called the Lipschitz constant

$$K = \sup \left[\frac{|f(x) - f(y)|}{|x - y|} \right]$$

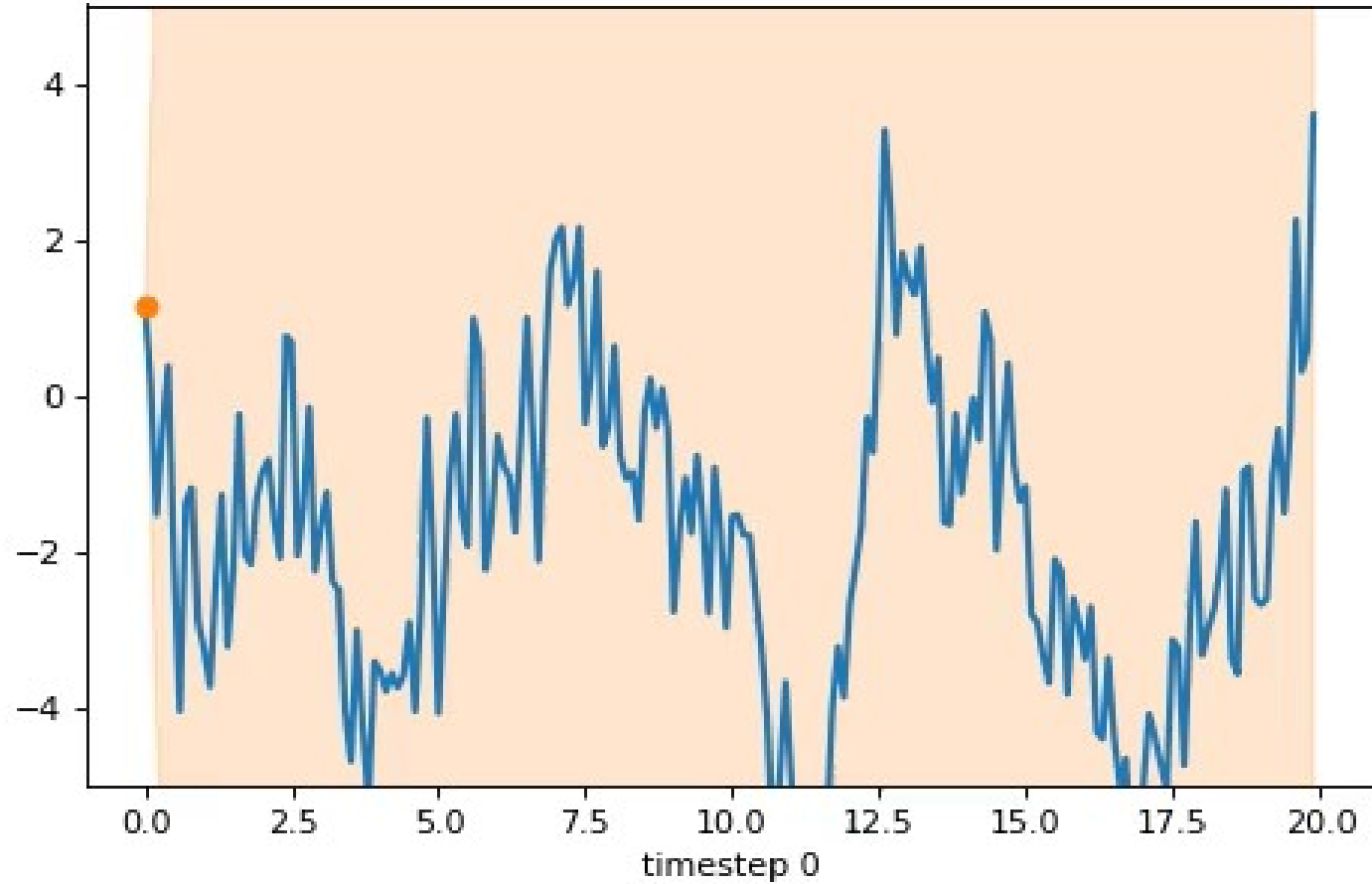
Where $\sup$ is the *supremum*, the smallest value that is larger than (or equal to) every value of this function

Small Lipschitz constant





Large Lipschitz constant





Analytical derivatives

- compute the derivative $f'(x) = \frac{d}{dx}f(x)$
- solve for the derivative being zero (i.e. solve x for $f'(x) = 0$). This finds all turning points or optima of the function.
- then check if any of the solutions has positive second derivative $f''(x) > 0$, which indicates the solution is a minimum.

Example: find the minimum of

$$f(x) = x^4 - 40x^2 - 100x$$

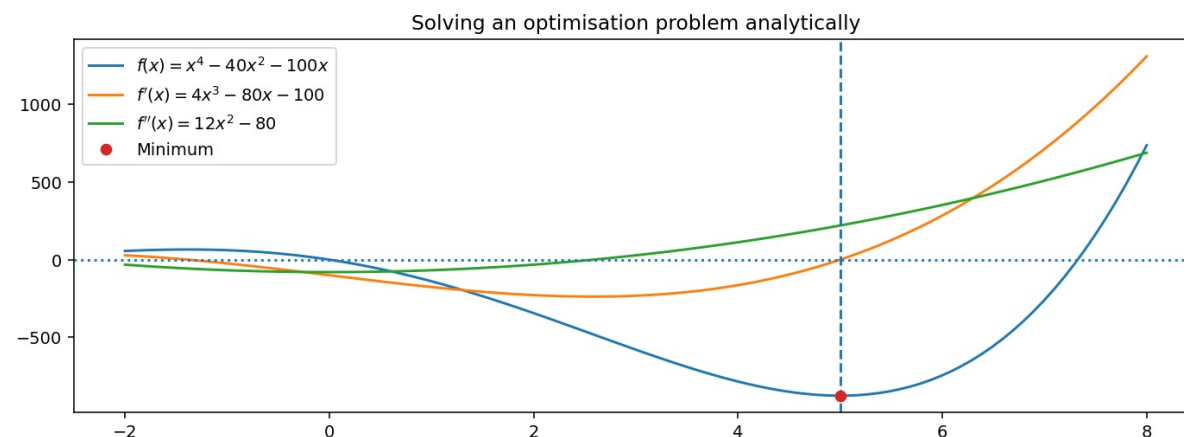
We can solve for:

$$f'(x) = 4x^3 - 80x - 100 = 0$$

and the second derivative is

$$f''(x) = 12x^2 - 80$$

and check the sign to find if we have a minimum.





Gradient : physical optimisation

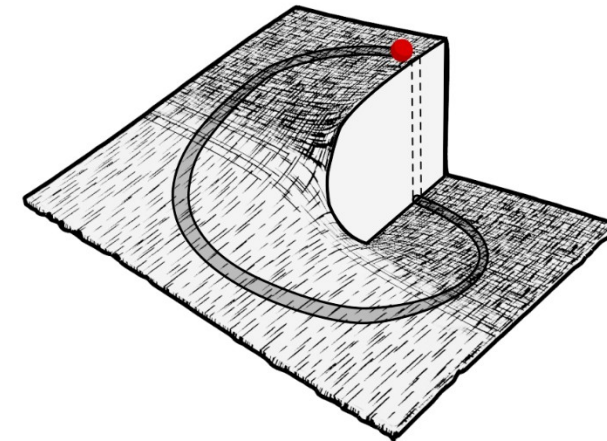
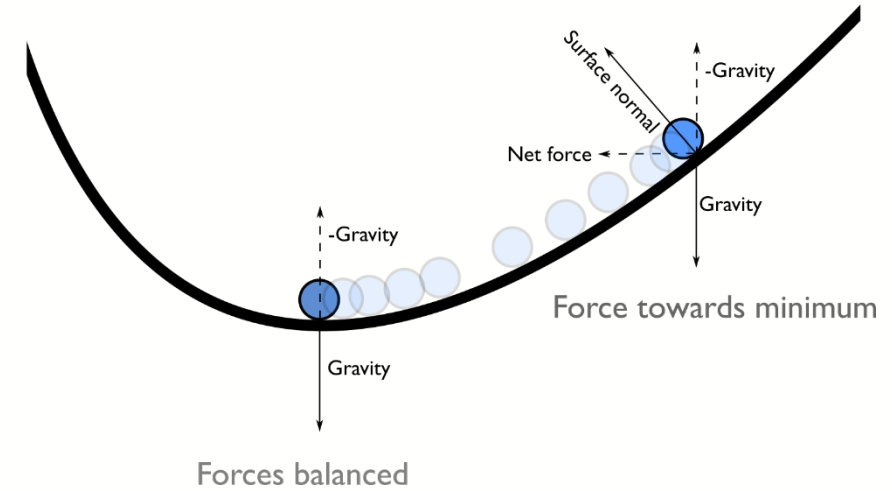
- Considering a ball rolling on a (smooth) surface, which represents the value of the objective function across a 2D domain (i.e. if we had a parameter vector θ with two elements).
- **Gravity** applies a force in *the direction of the steepest slope* of the surface, accelerating the ball in that direction.
- The **gradient** can be understood by thinking of it as a vector that points in the direction of the steepest ascent (or descent) on a surface representing a function



Gradient : physical optimisation

- This *gradient vector* always points in the direction of the steepest slope.
- Downhill is not always the shortest route, but can safely find a good local minima

Physical optimisation
(ignoring momentum)





Gradient descent

- The basic first-order algorithm is called **gradient descent** and it is very simple, starting from some initial guess $\theta^{(0)}$:

$$\theta^{(i+1)} = \theta^{(i)} - \delta \nabla L(\theta^{(i)})$$

where δ is a scaling hyperparameter -- the **step size**. The **step size** might be fixed, or might be chosen adaptively according to an algorithm like *line search*.

- This means is that the optimiser will make moves where the objective function drops most quickly.



Gradient descent

In simpler terms:

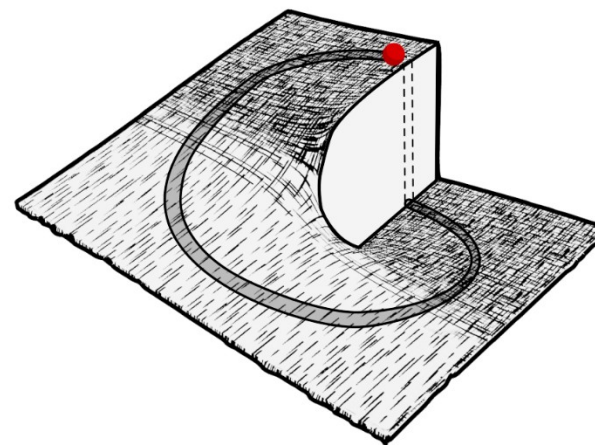
- starting somewhere $\theta^{(0)}$
- repeat:
 - check how steep the ground is in each direction $v = \nabla L(\theta^{(i)})$
 - move a little step δ in the steepest direction v to find $\theta^{(i+1)}$.

Notation note: $\theta^{(i)}$ does not mean the i th power of θ but just the i th θ in a sequence of iterations:

$\theta^{(0)}, \theta^{(1)}, \theta^{(2)}, \dots$

$$\nabla L(\theta) = \left[\frac{\partial L(\theta)}{\partial \theta_1}, \frac{\partial L(\theta)}{\partial \theta_2}, \dots, \frac{\partial L(\theta)}{\partial \theta_n} \right]$$

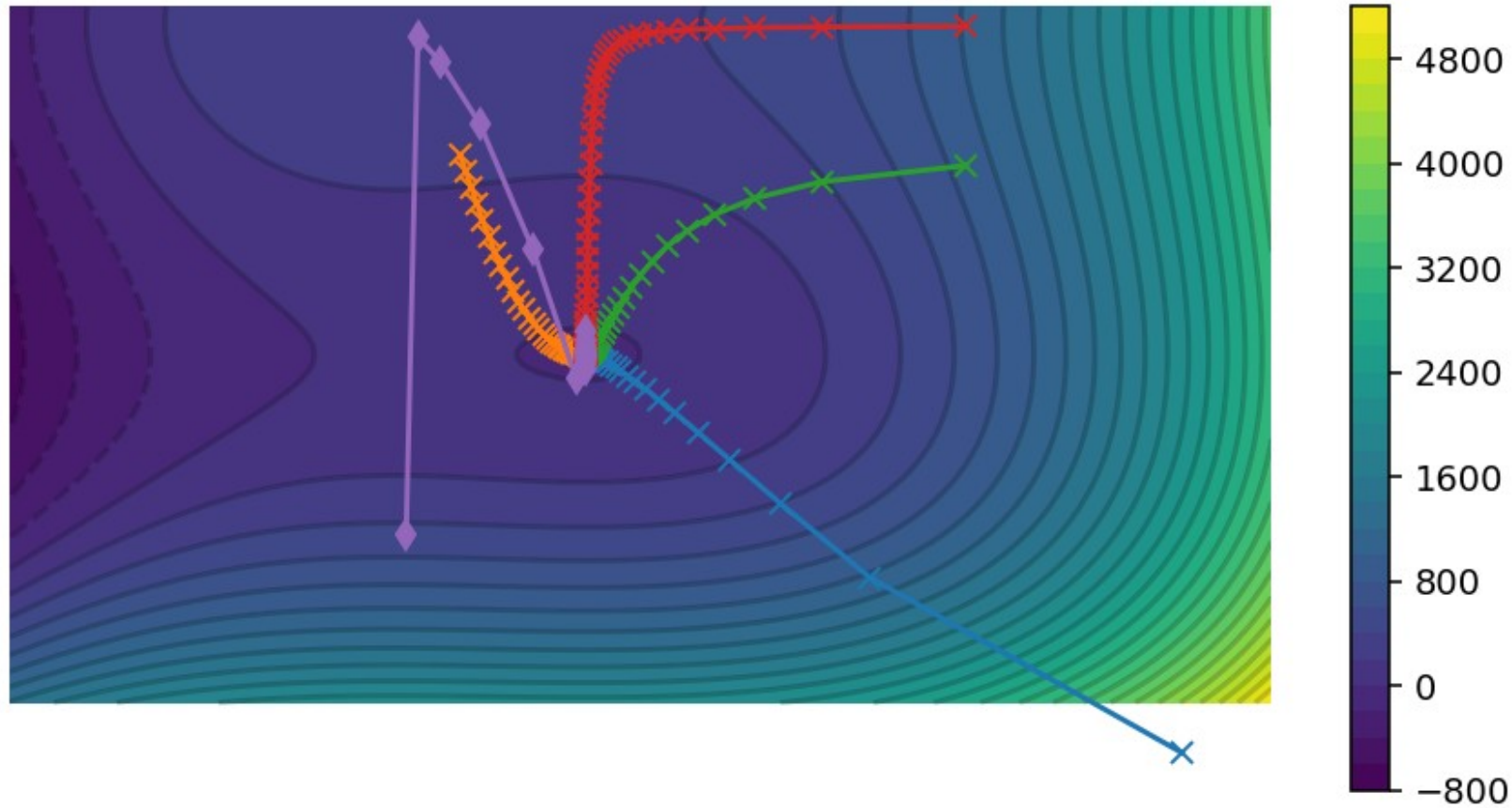
$$\theta^{(i+1)} = \theta^{(i)} - \delta \nabla L(\theta^{(i)})$$





Gradient descent from different initial conditions

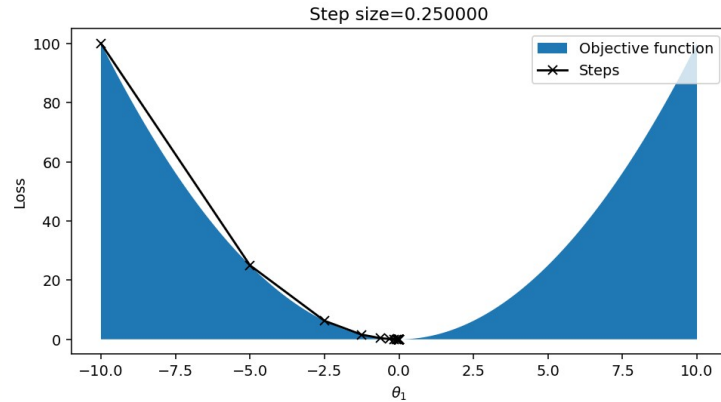
Gradient descent from various starting positions



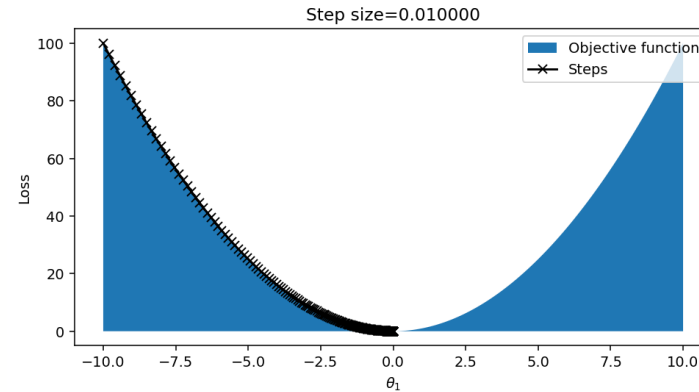


Why step size matters

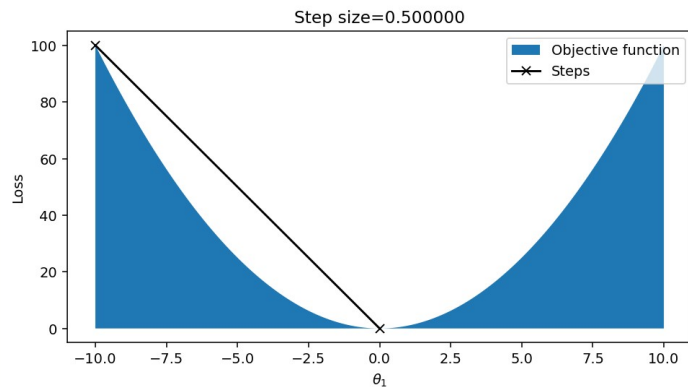
- Converged in 19 steps



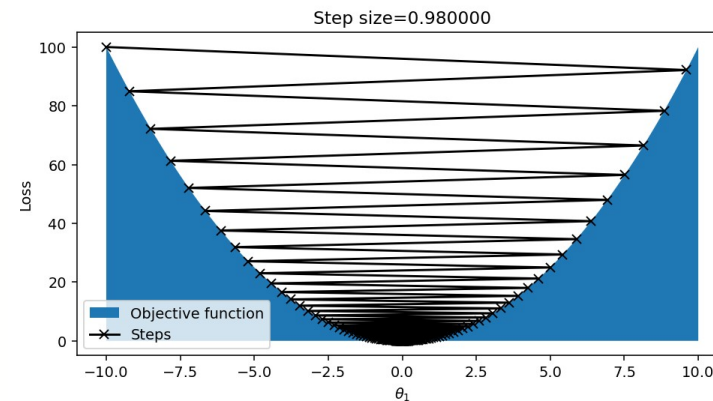
- Converged in 606 steps



- Converged in 2 steps



- Converged in 301 steps





Why not use numerical differences?

The definition of the derivative is well known:

$$\frac{d}{dx}f(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x-h)}{2h}$$

Why do we need the actual gradient? Why not just evaluate $f(x+h)$ and $f(x-h)$ at all points?

This is called *finite differences*.



Why not use numerical differences?

The definition of the derivative is well known:

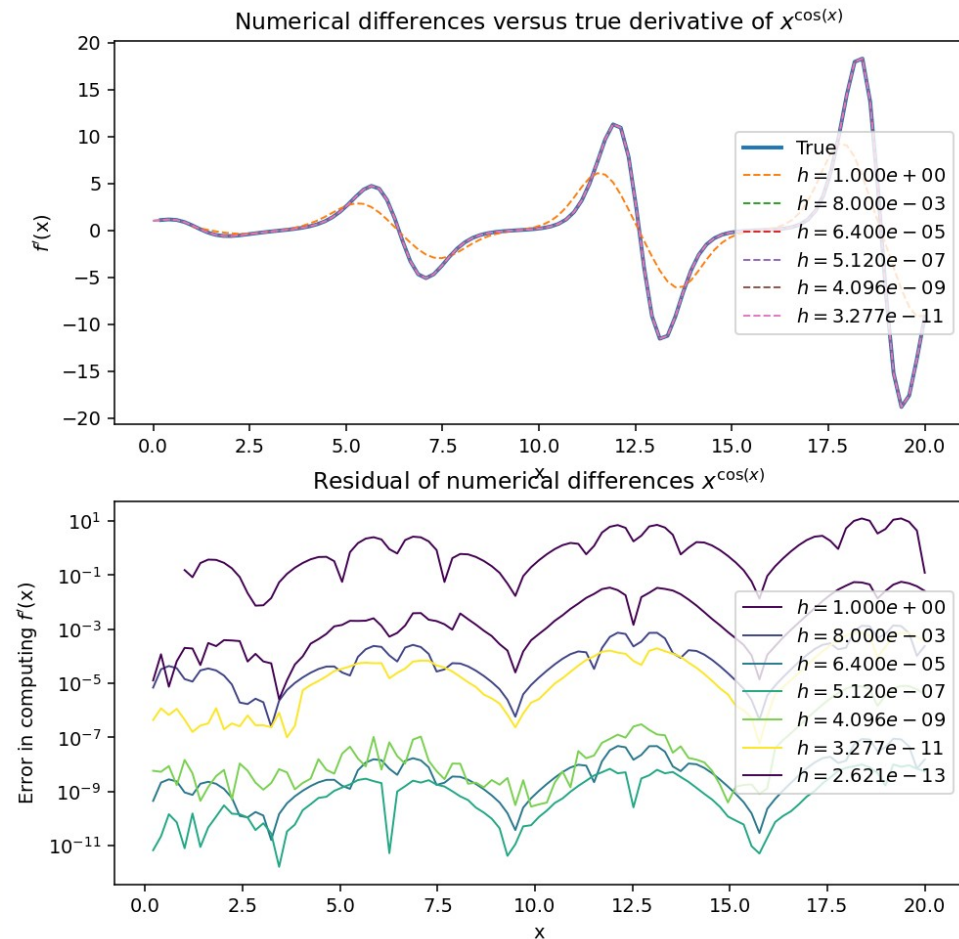
$$\frac{d}{dx}f(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x-h)}{2h}$$

Why do we need the actual gradient?
Why not just evaluate $f(x+h)$ and $f(x-h)$ at all points?

This is called *finite differences*.

Issues:

- Numerical stability
- Curse of dimensionality (again!)





Improving gradient descent

Gradient descent can be very efficient, and often **much** better than zeroth-order methods. However, it has drawbacks:

The gradient of the loss function $L'(\theta) = \nabla L(\theta)$ must be computable at any point θ . **Automatic differentiation** helps with this.

Gradient descent can get stuck in local minima. This is an inherent aspect of gradient descent methods, which will not find **global minima** except when the function is convex *and* the step size is optimal. **Random restart** and **momentum** are approaches to reduce sensitivity to local minima.

Gradient descent only works on **smooth**, '**differentiable**' objective functions. **Stochastic relaxation** introduces randomness to allow very steep functions to be optimised.

Gradient descent can be very slow if the objective function (and/or the gradient) is slow to evaluate. **Stochastic gradient descent** can massively accelerate optimisation if the objective function can be written as a simple sum of many subproblems.



Programming language advances

- **Vectorised programming**

- *Example:* NumPy, eigen, nd4j, J
- Provides native operations over **tensors**, potentially with **GPU** acceleration.

- **Differentiable programming**

- *Examples:* **autograd**, JAX, pytorch, tensorflow
- **Automatically differentiates** vectorised code, producing exact derivatives of tensor algorithms.

- **Probabilistic programming**

- *Examples:* pymc, stan, edward, Uber, Pyro, webppl
- Allows values to be uncertain, with (tensor, differentiable) random variables as first class values.



The magic behind modern advances in data science

- Automatic differentiation can take a function, usually written a *subset* of a full programming language, and automatically construct a function that evaluates the exact derivative at any given point. This makes it feasible to perform first-order optimisation.
- Some of the most common implementations of this technique are in deep learning software packages like *TensorFlow*, *Theano*, *Torch/PyTorch*.
- These provide a way of defining a **computational graph** (implicitly or explicitly) which defines the operations to be performed, and from which the corresponding derivative computation can be derived.
- These implementations tend to be focused on the operations used in neural networks, like matrix multiplication and elementwise nonlinear functions, and they tend not to include branching or iteration (or support very limited forms of conditional/looping expressions).



- Other software, like **autograd** provides automatic differentiation for virtually any NumPy code. The example below is from the autograd documentation. It is a drop in replacement which just "magically" estimates derivatives (although only some operations are supported).

```
import autograd.numpy as np
from autograd import grad, elementwise_grad

# just plain numpy
def tanh(x): # Define a function
    y = np.exp(-x)
    return (1.0 - y) / (1.0 + y)

# compute gradient
grad_tanh = grad(tanh) # Obtain its gradient function
print(grad_tanh(1.0)) # Evaluate the gradient at x = 1.0

0.39322386648296376
```

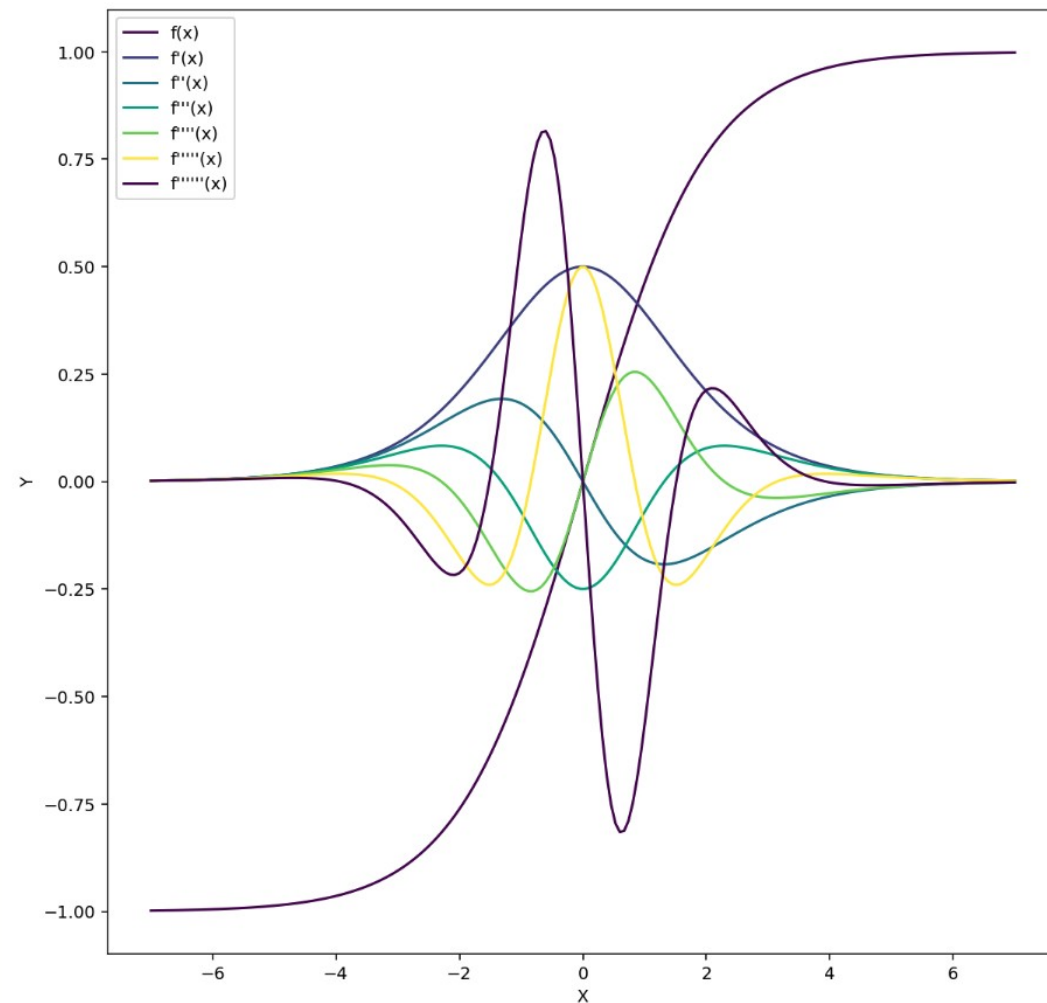
autograd has now evolved into **Google JAX**, probably the most promising machine learning library. JAX supports GPU and TPU computation with automatic differentiation. I'm not using it here because it is hard to install.



- Use Autograd

```
x = np.linspace(-7, 7, 200)
gr = elementwise_grad
fig = plt.figure(figsize=(10,10))
ax = fig.add_subplot(1,1,1)
ax.plot(x, tanh(x),
        x, gr(tanh)(x),           # first derivative
        x, gr(gr(tanh))(x),      # second derivative
        x, gr(gr(gr(tanh)))(x),  # third derivative
        x, gr(gr(gr(gr(tanh))))(x), # fourth derivative
        x, gr(gr(gr(gr(gr(tanh)))))(x), # fifth derivative
        x, gr(gr(gr(gr(gr(gr(tanh)))))(x)) # sixth derivative

ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.legend(["f(x)", "f'(x)", "f''(x)", "f'''(x)", "f''''(x)", "f'''''(x)", "f''''''(x)"])
```





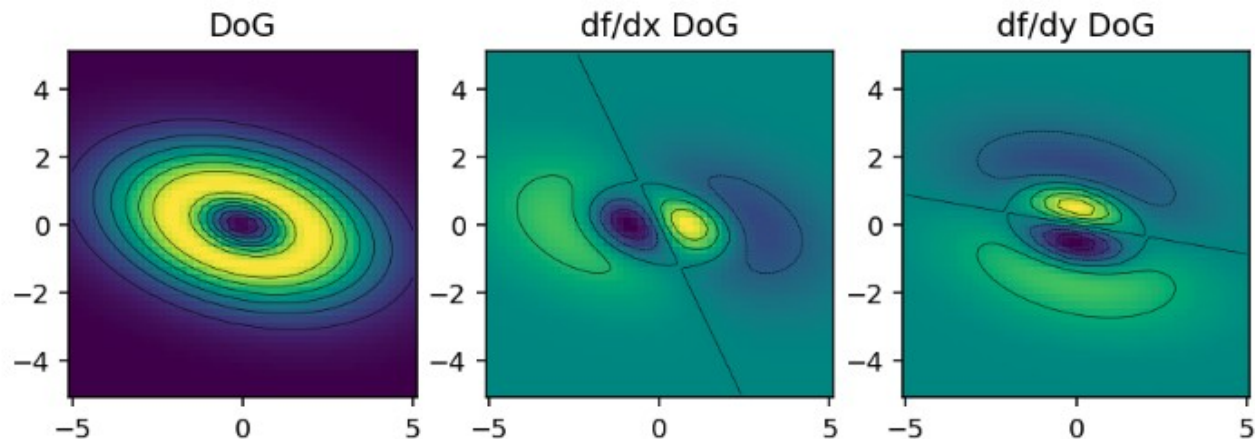
- **Vectorised exam**

```
f = np.array([[1, -0.2], [0.8, 1.5]])

def dog(x):
    x = x @ f
    return np.exp(-np.sum(x ** 2, axis=1) / 10.0) - np.exp(
        -np.sum(x ** 2, axis=1) / 2.0
    )

ddog = elementwise_grad(dog) # compute derivative via autograd
# that is *literally* all that is required
```

```
# plot the results
fig = plt.figure()
ax = fig.add_subplot(1, 3, 1)
show_fn(ax, dog, [-5, 5], 50)
ax.set_title("DoG")
ax = fig.add_subplot(1, 3, 2)
show_fn(ax, ddog, [-5, 5], 50, d=0)
ax.set_title("df/dx DoG")
ax = fig.add_subplot(1, 3, 3)
show_fn(ax, ddog, [-5, 5], 50, d=1)
ax.set_title("df/dy DoG")
```





Stochastic relaxation



How do animals evolve camouflage?

It either gets eaten by a predator who sees it or the predator does not see it → discontinuous fitness function!

Answer: although every *specific* case is a simple binary choice, it is *averaged* over many random instances, where the conditions will be slightly different

- (*maybe it is nearly dark, maybe the predator has bad eyes, maybe the weather is foggy*)

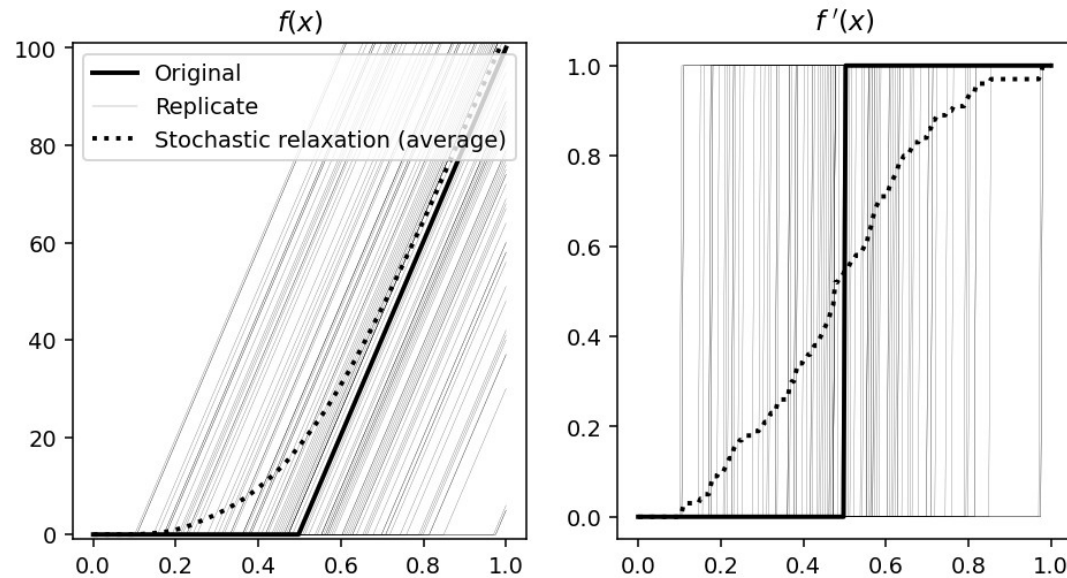
Averaging over all of those cases, some very minor change in colouring might offer an advantage.

This is called **stochastic relaxation**

See "**The Blind Watchmaker**" by *Richard Dawkins*, for a discussion of this topic.



Stochastic relaxation Example



Issue: Gradient descent can only be applied to function with continuous gradient (Lipschitz continuous)

If our loss function is a step function, we would not be able to optimise on it.

But if, for example we approximate a step functions with different randomly perturbed thresholds, then the average becomes continuous.

Stochastic Gradient Descent

Issue: Gradient descent evaluates the objective function and its gradient at each iteration before making a step. This can be very expensive to do, particularly when optimising function approximations with large data sets (e.g. in machine learning).

But: If the loss can be broken into small parts, for example a sum:

$$L(\theta) = \sum_i L_i(\theta)$$

Then, *because the gradient is a linear operator*, we can write:

$$\nabla L(\theta) = \sum_i \nabla L_i(\theta)$$

Then the optimiser can do gradient descent on randomly selected parts in turns which may be much faster and will (hopefully) converge to the same solution.

Stochastic Gradient Descent

For example, remember approximation problems

$$L(\theta) = \sum_i \|f(x_i; \theta) - y_i\|_2$$

Where the difference between the model's output and target is minimised, *summing over all training examples*, we can sum the gradients over all examples individually.

$$\nabla \sum_i \|f(x_i; \theta) - y_i\| = \sum_i \nabla \|f(x_i; \theta) - y_i\|$$

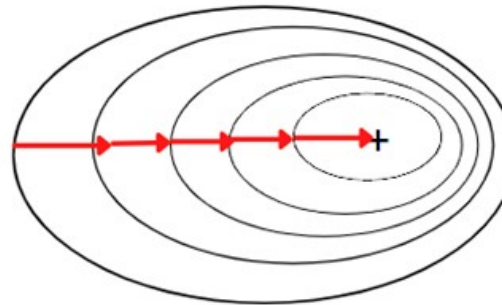
This means we can take random subsets of training examples (a **mini-batch**), and do gradient updates on each, requiring fewer evaluations per update.



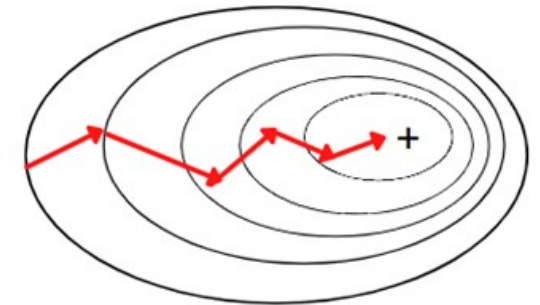
GD vs SGD vs mini-batch SGD

- In original SGD, you use only **ONE** training examples before updating the gradients.
- When the training set is large, SGD can be faster.
- But the parameters will “oscillate” toward the minimum rather than converge smoothly.
- Mini-batch is a middle ground between batch GD and SGD.

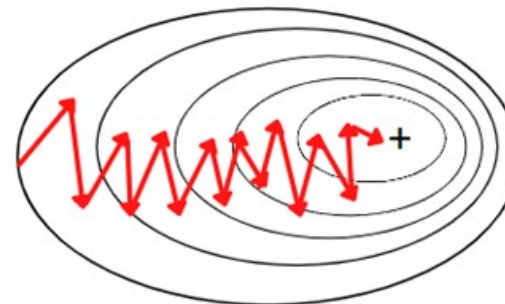
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent





Momentum

Because mini-batch gradient descent makes a parameter update after seeing just a subset of examples, the direction of the update has some variance, and so the path taken by mini-batch gradient descent will “oscillate” toward convergence.

Using momentum can reduce these oscillations. **Momentum** accumulates the gradient of the past steps to determine the direction to go.

The update rule is:

$$v_{t+1} = \gamma v_t + \eta \nabla_{\theta} L(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$

- v_t is the velocity vector representing the accumulated gradient.
- γ is the momentum factor, typically set to a value between 0.8 and 0.99.



Second-order derivatives

- If the first order derivatives represent the "slope" of a function, the second order derivatives represent the "curvature" of a function.

$$\nabla L(\theta) = \left[\frac{\partial L(\theta)}{\partial \theta_1}, \frac{\partial L(\theta)}{\partial \theta_2}, \dots, \frac{\partial L(\theta)}{\partial \theta_n} \right]$$

- For every parameter component θ_i the Hessian stores how the *steepness* of every other θ_j changes.

$$H(L) = \nabla \nabla L(\theta) = \nabla^2 L(\theta) = \begin{bmatrix} \frac{\partial^2 L(\theta)}{\partial \theta_1^2} & \frac{\partial^2 L(\theta)}{\partial \theta_1 \partial \theta_2} & \frac{\partial^2 L(\theta)}{\partial \theta_1 \partial \theta_3} & \cdots & \frac{\partial^2 L(\theta)}{\partial \theta_1 \partial \theta_n} \\ \frac{\partial^2 L(\theta)}{\partial \theta_2 \partial \theta_1} & \frac{\partial^2 L(\theta)}{\partial \theta_2^2} & \frac{\partial^2 L(\theta)}{\partial \theta_2 \partial \theta_3} & \cdots & \frac{\partial^2 L(\theta)}{\partial \theta_2 \partial \theta_n} \\ \cdots & & & & \\ \frac{\partial^2 L(\theta)}{\partial \theta_n \partial \theta_1} & \frac{\partial^2 L(\theta)}{\partial \theta_n \partial \theta_2} & \frac{\partial^2 L(\theta)}{\partial \theta_n \partial \theta_3} & \cdots & \frac{\partial^2 L(\theta)}{\partial \theta_n^2} \end{bmatrix},$$



- $\nabla^2 L(\theta)$ is the **Hessian matrix** of a (scalar) function of a vector, the equivalent of the **second** derivative for vector functions.
- Following our rule above, it's just the Jacobian of a **vector** valued function, so we know:

$\nabla^2 L(\theta)$ is matrix valued map $\mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}$

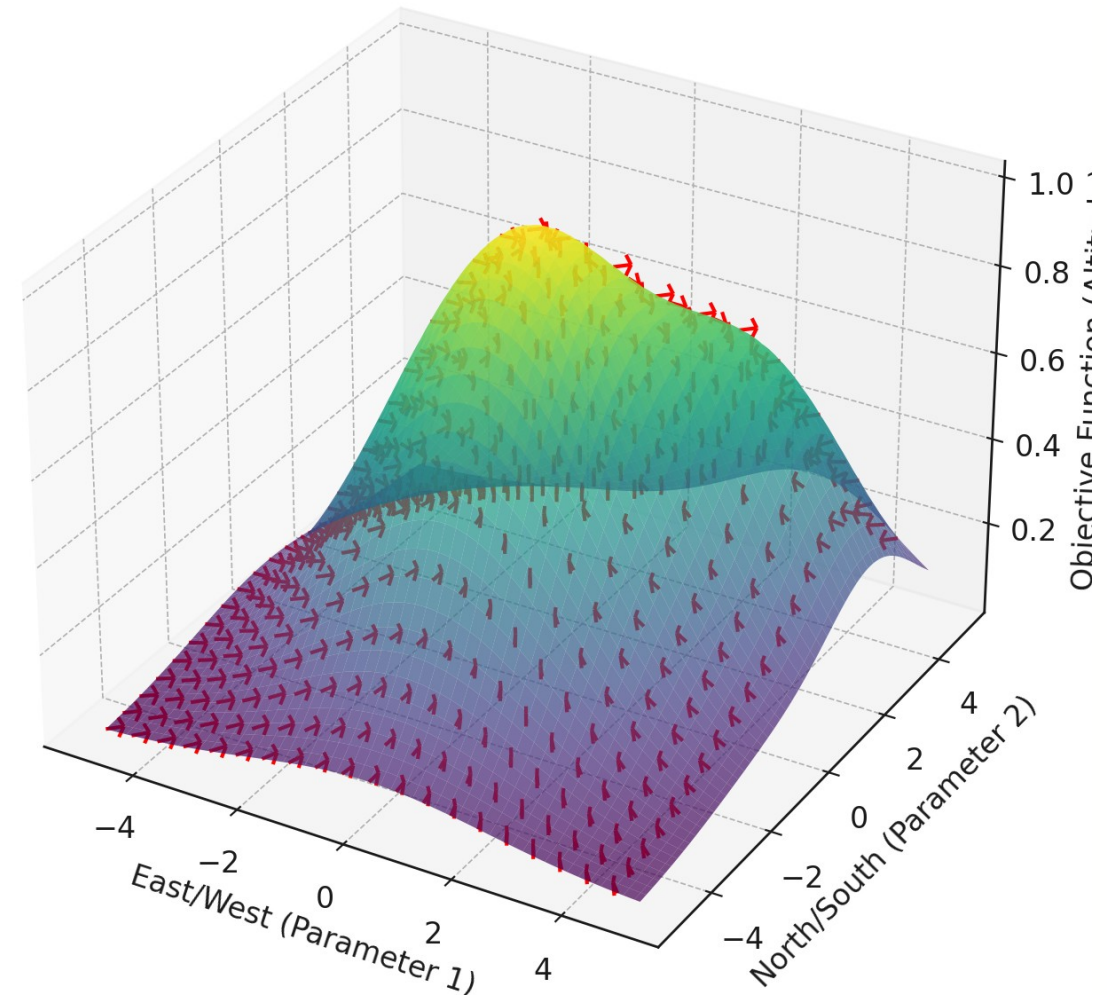
$$H(L) = \nabla \nabla L(\theta) = \nabla^2 L(\theta) = \begin{bmatrix} \frac{\partial^2 L(\theta)}{\partial \theta_1^2} & \frac{\partial^2 L(\theta)}{\partial \theta_1 \partial \theta_2} & \frac{\partial^2 L(\theta)}{\partial \theta_1 \partial \theta_3} & \cdots & \frac{\partial^2 L(\theta)}{\partial \theta_1 \partial \theta_n} \\ \frac{\partial^2 L(\theta)}{\partial \theta_2 \partial \theta_1} & \frac{\partial^2 L(\theta)}{\partial \theta_2^2} & \frac{\partial^2 L(\theta)}{\partial \theta_2 \partial \theta_3} & \cdots & \frac{\partial^2 L(\theta)}{\partial \theta_2 \partial \theta_n} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ \frac{\partial^2 L(\theta)}{\partial \theta_n \partial \theta_1} & \frac{\partial^2 L(\theta)}{\partial \theta_n \partial \theta_2} & \frac{\partial^2 L(\theta)}{\partial \theta_n \partial \theta_3} & \cdots & \frac{\partial^2 L(\theta)}{\partial \theta_n^2} \end{bmatrix},$$



Second-order derivatives

- Imagine I am on a hill.
 - The **altitude** I am at is the value of the **objective function**.
 - The **parameters** I can vary are my position **North/South** and **East/West**.
 - The **gradient vector** is the change in altitude as I take a step North or a step East, which are the two parameters. This is the local **steepness** at the place I am at on the hill.
 - The **Hessian** captures **how much steeper** the hill get **stepping Northwards** as I go **North** and *also* how much steeper the hill **gets Eastwards** as I step **North**; similarly for stepping East. So there is a **2x2 matrix** describing these changes in steepness.

Objective Function with Gradient Vectors





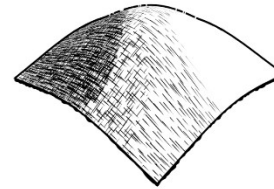
Eigenvalues of the Hessian & types of critical points

The Hessian matrix captures important properties about the **type of critical point** that we saw in the previous lecture. In particular, the **eigenvalues** of the Hessian tell us what kind of point we have.

- If all eigenvalues are all strictly positive, the matrix is called **positive definite** and the point is a **minimum**.
- If all eigenvalues are all strictly negative (**negative definite**) and the point is a **maximum**.
- If eigenvalues have mixed sign the point is a **saddle point**.
- If the eigenvalues are all positive/negative, but with some zeros, the matrix is **semidefinite** and the point is plateau/ridge.

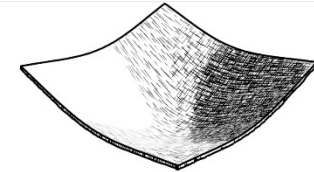
Maxima

Gradient decreases in all dimensions



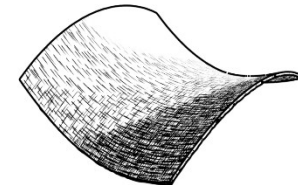
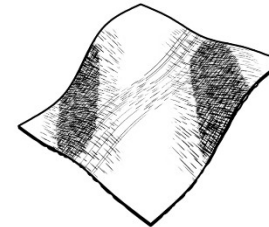
Minima

Gradient increases in all dimensions



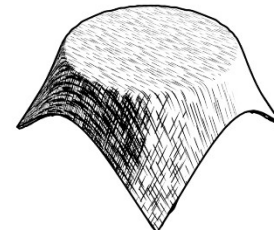
Saddle points

Gradient has different signs in different dimensions



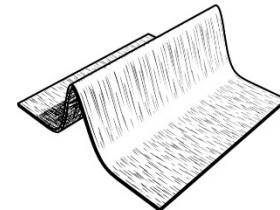
Plateau

Gradient zero in all dimensions



Ridge

Gradient zero in one dimension





Newton-Conjugate Gradient

- **Newton's Method:** Uses second-order information (Hessian matrix) to account for curvature.

$$w_{new} = w - H^{-1} \nabla f(w)$$

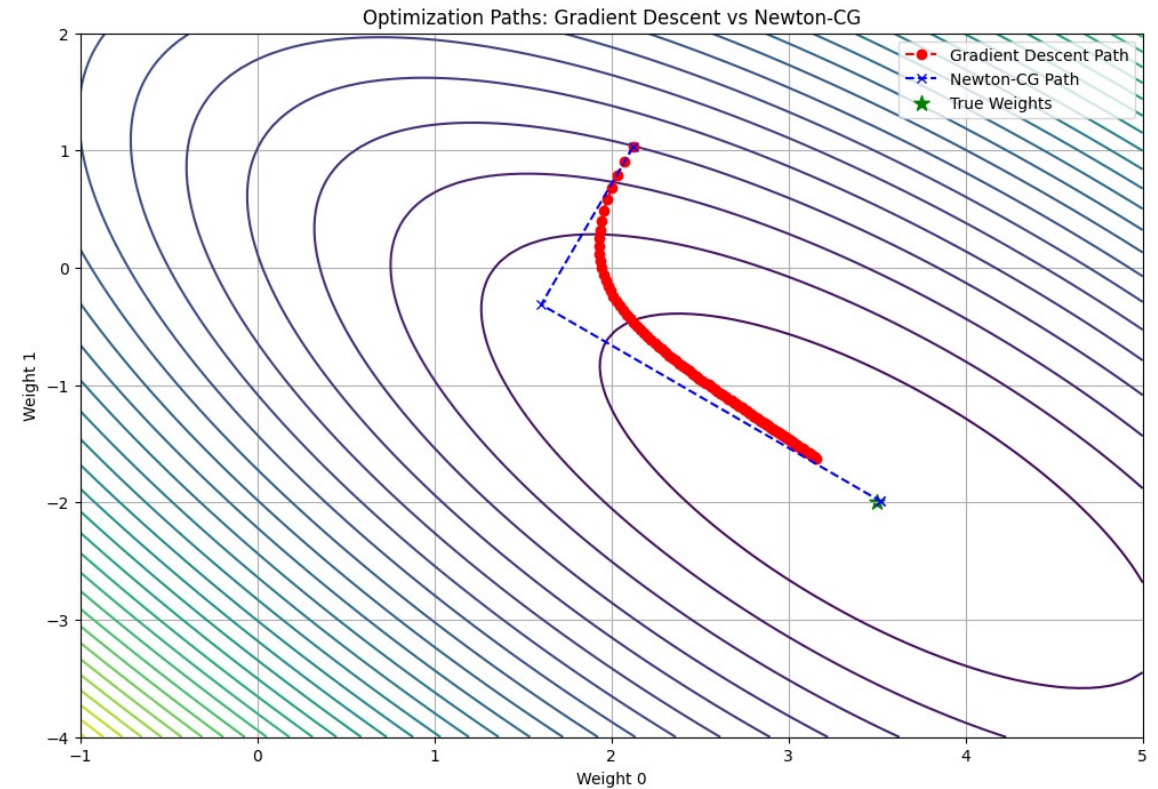
- **Conjugate Gradient (CG):**

Efficiently solves linear systems involving the Hessian without direct inversion.

- Newton-CG typically converges faster than gradient descent because it uses second-order information (the Hessian), making it more effective, especially for problems with high curvature.
- A full computation of the Hessian matrix (second-order derivatives) is both memory-intensive and computationally costly for large datasets. Evaluating the Hessian matrix requires d^2 computations, and d^2 storage.

Why Newton-CG converges faster?

- Adjust the step size and direction more accurately
- Near a local minimum, Newton's method converges quadratically—meaning the error decreases exponentially with each iteration, requiring fewer steps to achieve a highly accurate solution.



See: [IDSS - optimisation III - lecture codes.ipynb](#)



Use the right algorithm

- If you know the problem is **least-squares** use a specialised least-squares solver. You might be able to solve directly, for example with the pseudo-inverse.
- If you know the problem is **convex**, use a convex solver. This is radically more efficient than any other choice if its applicable.
- If you know the derivatives of the objective function, or can compute them using automatic differentiation, use a **first-order** method (or second order, if you can)
- If you don't know any of these things, use a general purpose **zeroth-order** solver like **simulated annealing** or a **genetic algorithm**.



Use the right algorithm

- If you know the problem is **least-squares** use a specialised least-squares solver. You might be able to solve directly, for example with the pseudo-inverse.
- If you know the problem is **convex**, use a convex solver. This is radically more efficient than any other choice if its applicable.
- If you know the derivatives of the objective function, or can compute them using automatic differentiation, use a **first-order** method (or second order, if you can)
- If you don't know any of these things, use a general-purpose **zeroth-order** solver like **simulated annealing** or a **genetic algorithm**.



University
of Glasgow

Thank you for you attention!

Contact: Nicolas.Pugeault@Glasgow.ac.uk

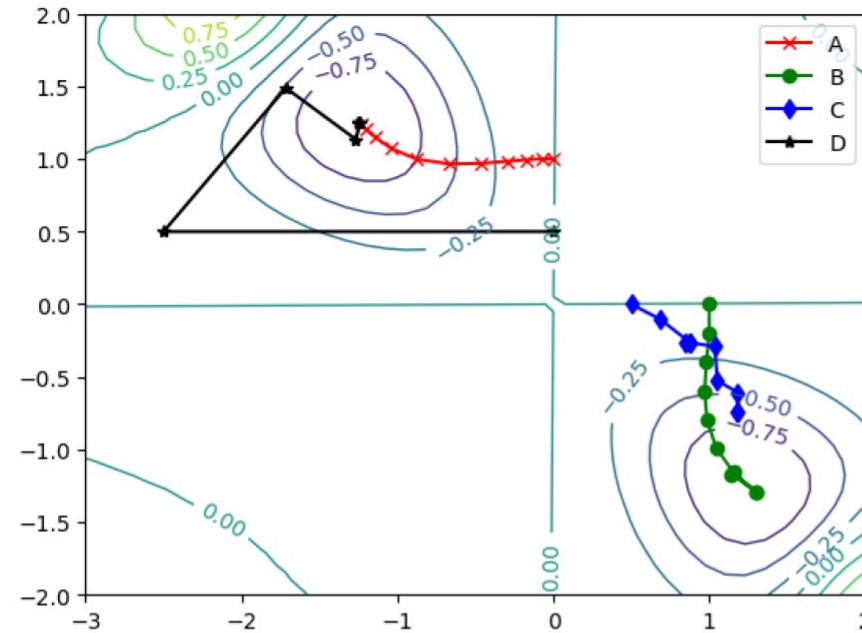


University
of Glasgow

Example Optimisation exam question

December 2023 Exam paper

2. (a) Look carefully at the plot in the image below.

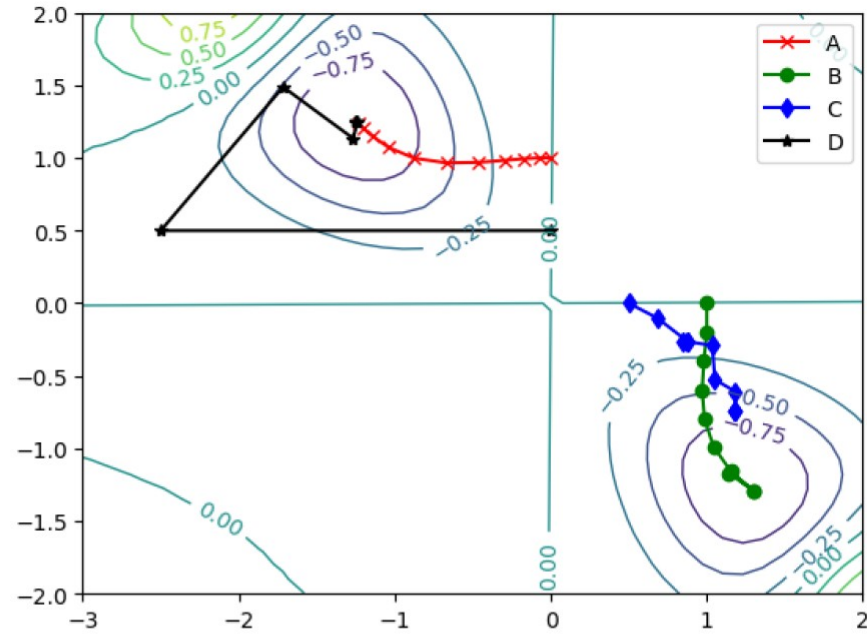


This is a contour plot of the loss function you are trying to minimise.

Explain how you would best qualify the loss function at the following points, and what you would expect the gradient to be and why:

- at $(-1, -1)$
- at $(-1, 1)$
- at $(-2, 1.5)$

(comment on both the direction and magnitude of the gradient)

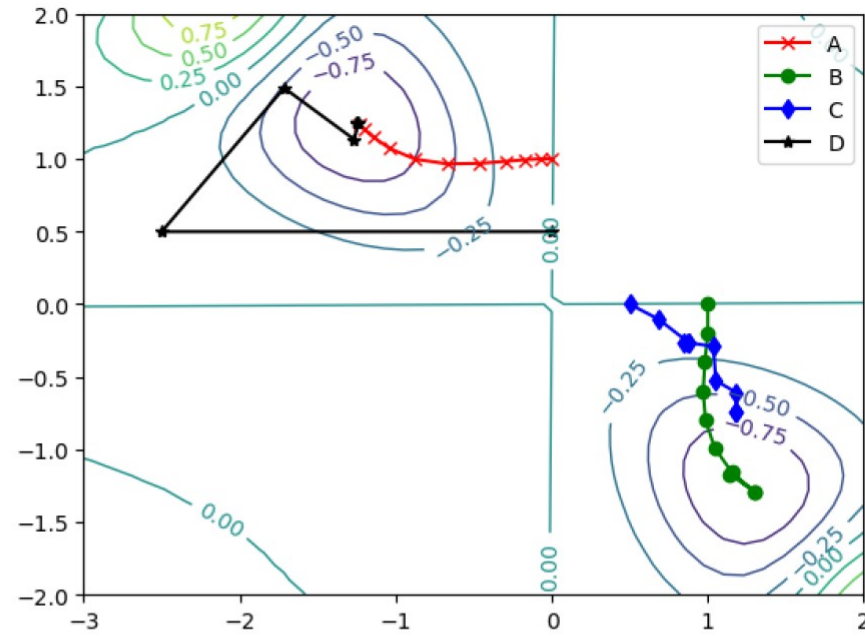


(b) The red (A), green (B), blue (C) and black (D) lines show the trajectories of four iterative optimisation algorithms that a colleague of yours attempted (each had a different starting point).

Looking at the graphs:

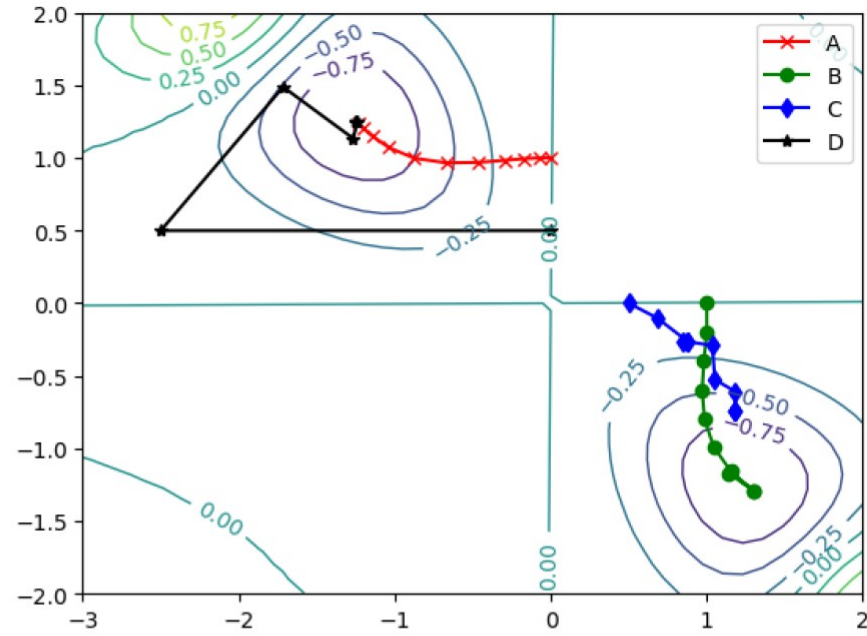
- Which one of the three is most likely to be a *gradient descent* algorithm?
- Which one is most likely to be a *hill-climbing* algorithm?
- Which one is most likely to be a *Newton method*?

Explain why in each case.



- (c) What could be an issue when optimising this loss with iterative algorithms (for example gradient descent)? What solution could you suggest to mitigate this issue? Looking at the graph, do you think this is likely to be a problem for your application?

[5]



(d) Assume that you want your optimisation to *not* find the (local) minimum to the bottom right of the graph. Explain in details how to achieve this.

[3]